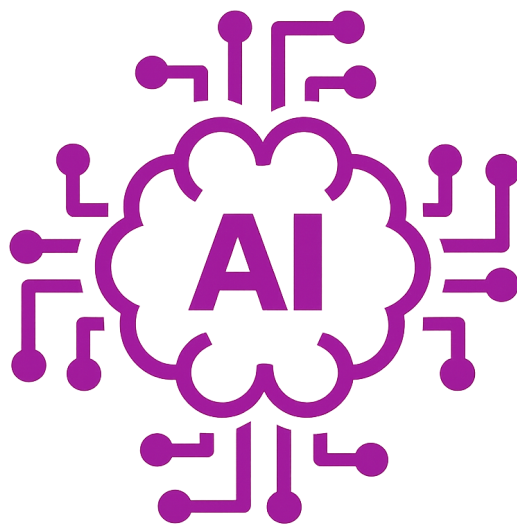


ML / DL : PBL 03

«Разбиение клиентов на группы способом K - Means»



SK Shieldus

Rookies, 30-й набор (личный номер 14)

Ким Джэ-Ук

Решение PBL 03

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score, silhouette_samples
from sklearn.model_selection import train_test_split

# загружаем данные и смотрим, как они устроены
customer_df = pd.read_csv("20260115_141658_mall_customers.csv")

# бегло осматриваем таблицу
print(f'В этой таблице число строк {customer_df.shape[0]} и '
      f'число столбцов {customer_df.shape[1]}')
print("-"*30)
print("Выведем все столбцы, какие тут есть\n"
      f'{'', ' '.join(customer_df.columns.values.astype(str)[:])}')
print("-"*30)
# посмотрим на первые семь строк
customer_df.head(7)
# в Quarto таблица DataFrame выводится через средства Jupyter,
# поэтому она получается опрятной и без отдельной настройки HTML
```

В этой таблице число строк 200 и число столбцов 5

Выведем все столбцы, какие тут есть

CustomerID, Gender, Age, Annual Income (k\$), Spending Score (1-100)

	CustomerID	Gender	Age	Annual Income (k\$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40
5	6	Female	22	17	76
6	7	Female	35	18	6

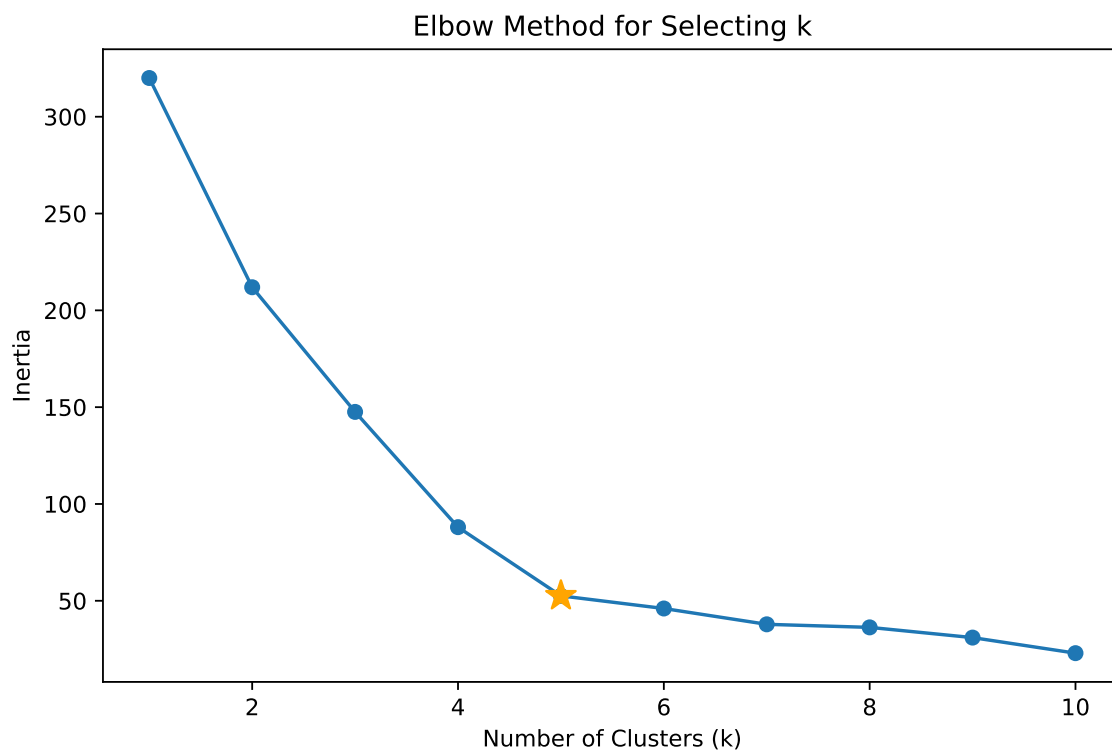
```
# посмотрим, есть ли пропуски в каждом столбце
print(customer_df.isna().sum().to_dict())
```

```
{'CustomerID': 0, 'Gender': 0, 'Age': 0, 'Annual Income (k$)': 0, 'Spending Score (1-100)': 0}
```

По выводу кода выше можно убедиться, что пропусков нет совсем .

```
# отбираем переменные для разбора, берём только годовой доход
# и оценку трат, и они пойдут на вход разбиению на группы
feature_df = customer_df[["Annual Income (k$)", "Spending Score (1-100)"]]
# обучение здесь идёт без ответов, но мы всё равно делим данные
# на обучающую и проверочную части, чтобы проверить, устойчиво ли
# найденные центры групп ложатся на данные, которых модель не видела
# делим данные в отношении 8 к 2
train_features, test_features = train_test_split(feature_df, test_size=0.2,
                                                  random_state=42)

# приводим переменные к одному масштабу, ведь K - Means
# работает на расстояниях
standardizer = StandardScaler()
# находим меру по обучающим данным и сразу их преобразуем
train_scaled = standardizer.fit_transform(train_features)
# проверочные данные преобразуем по той же самой мере
test_scaled = standardizer.transform(test_features)
# ищем лучшее число групп способом локтя, для этого меняем k
# от 1 до 10 и каждый раз считаем меру разброса внутри групп
inertia_values = []
cluster_candidates = range(1, 11)
for cluster_count in cluster_candidates:
    kmeans_model = KMeans(n_clusters=cluster_count, random_state=42)
    # обучаем K-Means на обучающих данных
    kmeans_model.fit(train_scaled)
    # сохраняем меру разброса внутри групп
    inertia_values.append(kmeans_model.inertia_)
# строим рисунок способа локтя, то есть смотрим, как мера разброса
# меняется вместе с k
plt.figure(figsize=(8, 5))
plt.plot(cluster_candidates, inertia_values, marker="o")
# число групп (k)
plt.xlabel("Number of Clusters (k)")
# мера разброса внутри групп
plt.ylabel("Inertia")
plt.title("Elbow Method for Selecting k")
# отдельно отмечаем точку k = 5 жёлтой звёздочкой, а так как
# cluster_candidates идёт от 1 до 10, порядковый номер будет 5 - 1
k_target = 5
plt.scatter([k_target], [inertia_values[k_target - 1]], c="orange",
            marker="*", s=185, zorder=5)
plt.show()
```



Разбор плотности данных (Cohesion) & выбор k по рисунку способа локтя

В этом разборе **плотность** означает то, насколько тесно данные одной и той же группы собраны вокруг её центра, и в K - Means мы измеряли её косвенно через **меру разброса внутри групп (inertia , сумма квадратов расстояний внутри группы)**. В коде эта мера была посчитана для $k = 1 \sim 10$, а затем окончательное k было выбрано там, **где убывание меры разброса становится пологим ($k = 5$)**. Так сделано потому, что **обычно с ростом k групп становится больше, каждая группа и её мера разброса уменьшаются, а плотность растёт, но при этом растёт и сложность модели, и опасность чрезмерной подгонки**.

Кроме того, дальше можно будет увидеть, что на рисунке с обучающими данными точки каждой группы довольно заметно собраны вокруг её центра, и это указывает на то, что обучение сошлось в сторону уменьшения расстояний внутри групп, а это и есть цель K - Means.

```
# обучаем окончательную модель, по рисунку способа локтя берём k = 5
chosen_k = 5
final_kmeans = KMeans(n_clusters=chosen_k, random_state=42)
# обучаем окончательную модель на обучающих данных
final_kmeans.fit(train_scaled)

train_labels = final_kmeans.labels_
train_silhouette_avg = silhouette_score(train_scaled, train_labels)
print(f"Средняя оценка силуэта на обучающих данных {train_silhouette_avg:.2f}")
print("""*30)
print("- - ""*30)

# строим рисунок оценок силуэта, для этого считаем оценку
# для каждой отдельной точки
train_silhouette_values = silhouette_samples(train_scaled, train_labels)

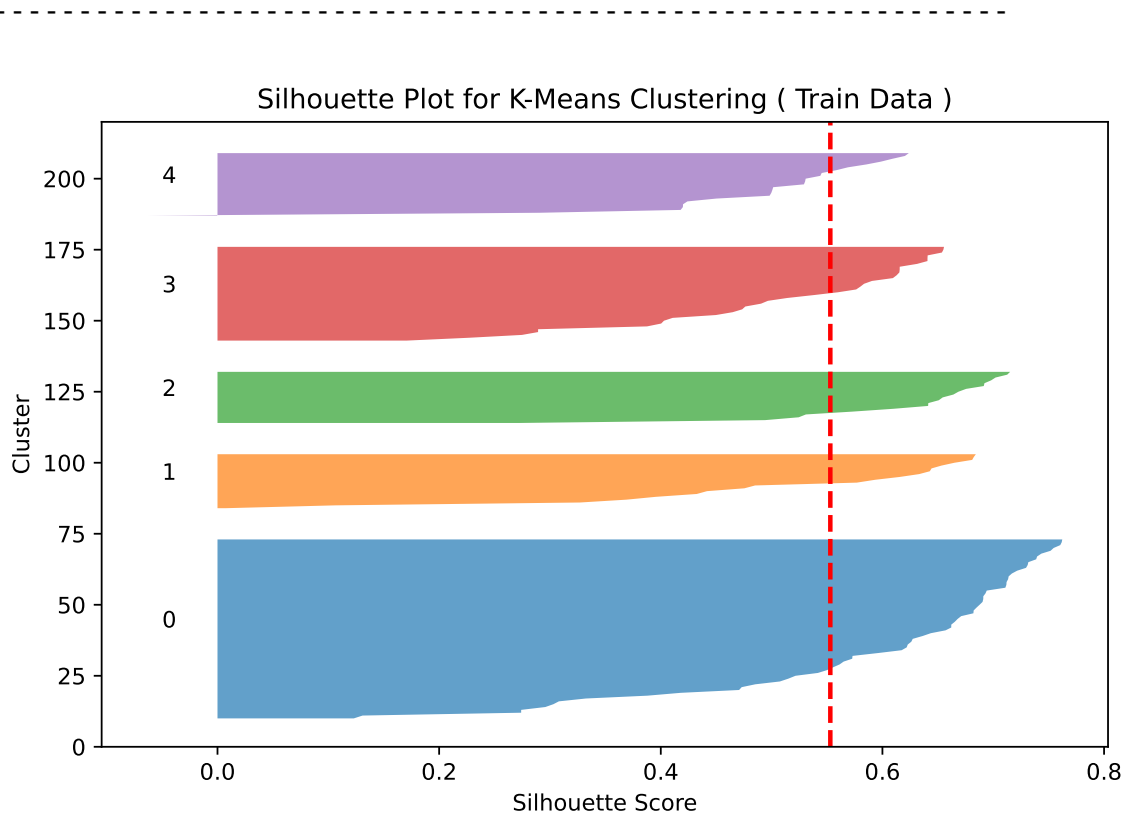
# готовим рисунок
plt.figure(figsize=(8, 5))
y_lower = 10

# проходим по каждой группе
for cluster_id in range(chosen_k):
    ith_vals = train_silhouette_values[train_labels == cluster_id]
    ith_vals.sort()
    size_cluster = ith_vals.shape[0]
    y_upper = y_lower + size_cluster
    # рисуем полосу для каждой группы
    plt.fill_between(np.arange(y_lower, y_upper), 0, ith_vals, alpha=0.7)
    plt.text(-0.05, y_lower + 0.5 * size_cluster, str(cluster_id))
    y_lower = y_upper + 10

# настройки рисунка
```

```
plt.axvline(x=train_silhouette_avg, color="red", linestyle="--", linewidth=2)
plt.title("Silhouette Plot for K-Means Clustering ( Train Data )")
plt.xlabel("Silhouette Score")
plt.ylabel("Cluster")
plt.ylim(0, y_lower)
plt.show()
```

Средняя оценка силуэта на обучающих данных 0.55



Разбор оценки силуэта (**Silhouette Score**) по рисунку для обучающих (**Train**) данных

Оценка силуэта считается для каждого наблюдения и сразу учитывает и среднее расстояние внутри своей группы (склонность к плотности), и среднее расстояние до ближайшей чужой группы (склонность к разделению). Чем это значение больше, тем сильнее наблюдение принадлежит своей группе (лежит близко) и тем лучше оно отделено от чужих групп (лежит далеко). На этих обучающих данных **средняя оценка силуэта** равна **0.55**, и на рисунке она показана опорной линией из **красного пунктира**.

Если всмотреться в рисунок, у большинства наблюдений оценка силуэта

складывается **выше 0.5**, и это указывает на то, что разделение между группами в целом держится на вполне осмысленном уровне. Однако у разных групп длина полос (размах оценок силуэта) и толщина (число наблюдений в группе) оказываются разными, поэтому **считать качество групп одинаковым для всех нельзя**.

А именно, у группы **номер 0** по сравнению с остальными **полоса явно самая толстая**, а значит, **в ней больше всего наблюдений**. Кроме того, у неё **очень широкий размах оценок силуэта**. Это указывает на то, что внутри группы номер 0 вместе оказались и наблюдения, лежащие сравнительно близко к границе групп (с низкой оценкой), и наблюдения, прочно принадлежащие своей группе (с высокой оценкой).

В целом **среднюю оценку силуэта 0.55 на обучающих данных** можно толковать так, что **строение групп в целом неплохое**, но **сила разделения и наличие наблюдений у границы у разных групп разные**.

```

# обученной моделью предсказываем группы для проверочных данных
test_labels = final_kmeans.predict(test_scaled)

# считаем оценку силуэта на проверочных данных, чтобы посмотреть,
# насколько группы отделены друг от друга
test_silhouette_avg = silhouette_score(test_scaled, test_labels)

print(f"Средняя оценка силуэта на проверочных данных {test_silhouette_avg:.2f}")

# пунктирная черта делает вывод про среднюю оценку силуэта
# на проверочных данных заметнее и удобнее для чтения
print("- - " * 30)

# считаем оценку силуэта, то есть меру пригодности группы,
# для каждой точки проверочных данных
test_silhouette_values = silhouette_samples(test_scaled, test_labels)

# задаём размер рисунка
plt.figure(figsize=(8, 5))
# место по оси y, с которого начнётся первая группа, с отступом
y_lower = 10

# показываем разброс оценок силуэта по каждой группе
for cluster_id in range(chosen_k):
    # берём оценки силуэта у точек этой группы
    ith_vals = test_silhouette_values[test_labels == cluster_id]
    # расставляем значения по возрастанию, чтобы полоса шла ровно
    ith_vals.sort()

    # число точек в этой группе
    size_cluster = ith_vals.shape[0]
    # где по оси y закончится место этой группы
    y_upper = y_lower + size_cluster

    # закрашиваем оценки силуэта в виде горизонтальной полосы
    plt.fill_betweenx(np.arange(y_lower, y_upper), 0, ith_vals, alpha=0.7)
    # подписываем номер группы рядом с полосой
    plt.text(-0.05, y_lower + 0.5 * size_cluster, str(cluster_id))

    # сдвигаем место по оси y для следующей группы и добавляем промежуток
    y_lower = y_upper + 10

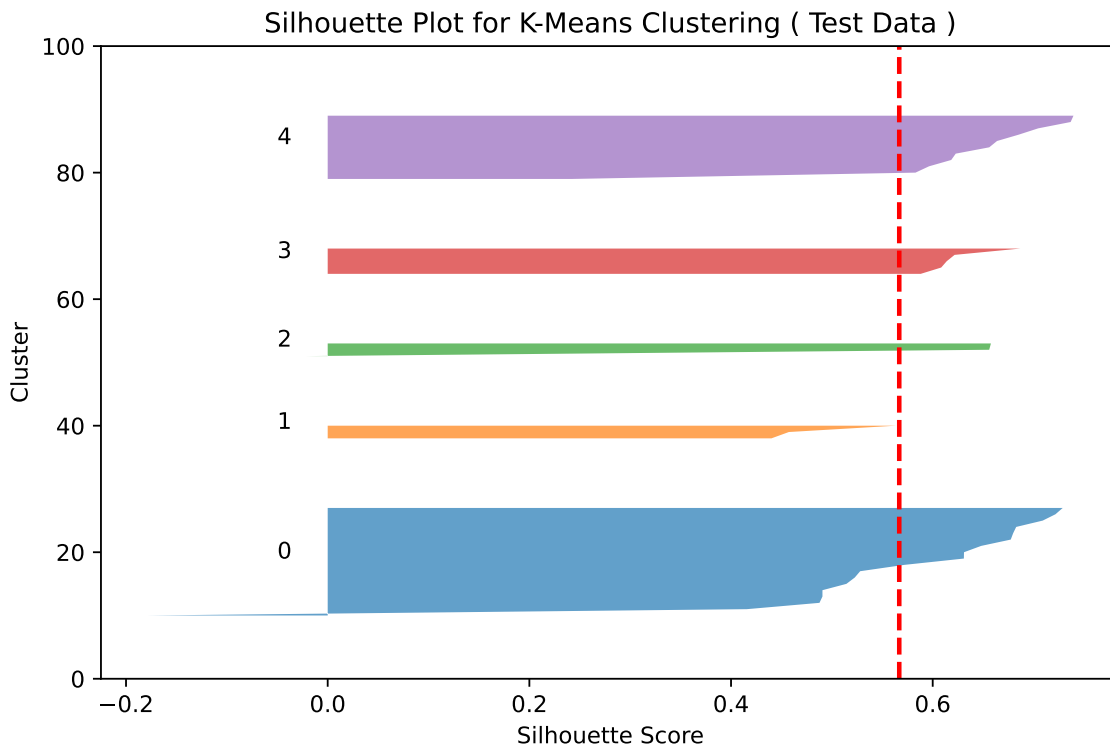
# оформляем рисунок
plt.axvline(x=test_silhouette_avg, color="red", linestyle="--", linewidth=2)
plt.title("Silhouette Plot for K-Means Clustering ( Test Data )")

```



```
plt.xlabel("Silhouette Score")
plt.ylabel("Cluster")
plt.ylim(0, y_lower)
plt.show()
```

Средняя оценка силуэта на проверочных данных 0.57

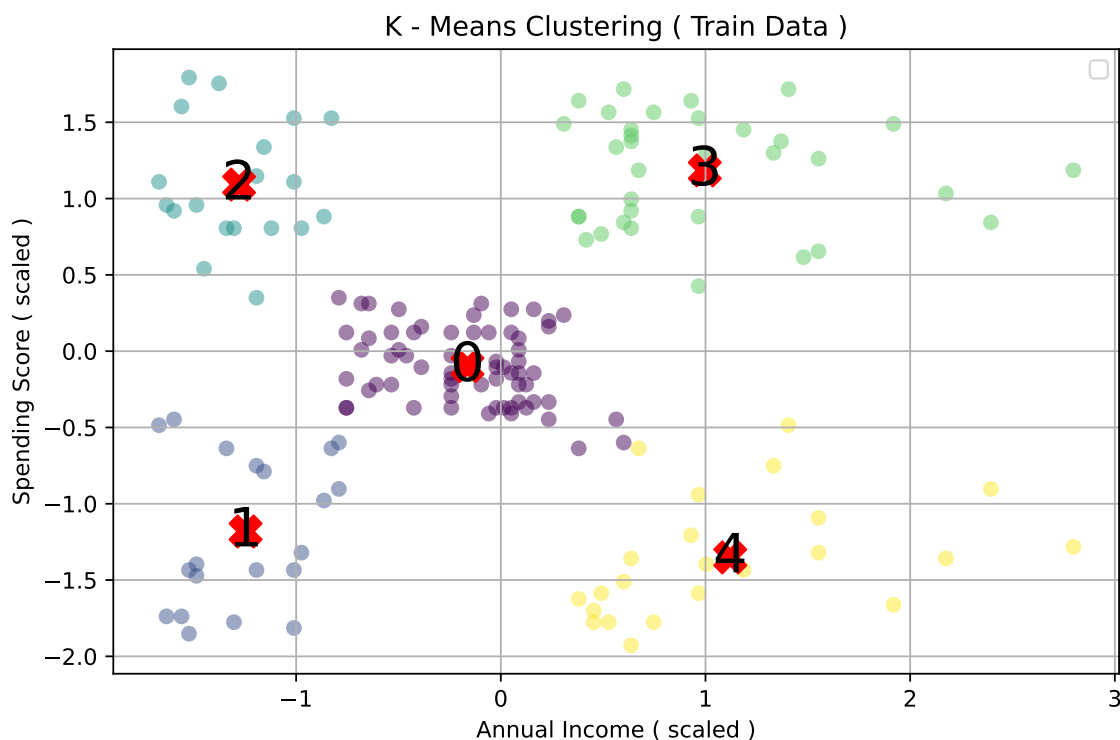


Разбор оценки силуэта (**Silhouette Score**) по рисунку для проверочных (**Test**) данных

На этих проверочных данных **средняя оценка силуэта** равна **0.57**, а значит, разбиение на группы вышло осмысленным, и на рисунке эта величина показана **красным пунктиром**. Если всмотреться в рисунок, у **группы номер 4** полоса заметно тянется вправо от средней линии, поэтому её можно толковать как **группу, у которой разделение держится сравнительно сильно**. Напротив, у **группы номер 0** в **небольшом количестве** видны **значения ниже нуля (отрицательные)**, и это указывает на возможное присутствие точек у границы или точек, отнесённых к группе неустойчиво. (← то же самое относится и к разбору оценок силуэта на обучающих данных) Вместе с тем у **группы номер 1** точки чаще лежат слева от средней линии, а это значит, что часть её наблюдений **может быть отделена от чужих групп сравнительно слабо**. В целом **среднюю оценку 0.57 на проверочных данных** можно толковать так, что **строение групп в**

целом воспроизвелось, но сила разделения и наличие точек у границы по группам не одинаковы.

```
# рисунок с результатом разбиения на обучающих данных вместе с центрами
plt.figure(figsize=(8, 5))
plt.scatter(train_scaled[:, 0], train_scaled[:, 1], c=final_kmeans.labels_,
            cmap="viridis", alpha=0.5)
plt.scatter(*final_kmeans.cluster_centers_.T, s=200, c="r", marker="X")
for i,(cx,cy) in enumerate(final_kmeans.cluster_centers_): plt.text(cx, cy, i, fontsize=24, ha="center", va="center")
plt.title("K - Means Clustering ( Train Data )")
plt.xlabel("Annual Income ( scaled )")
plt.ylabel("Spending Score ( scaled )")
plt.legend()
plt.grid(True)
plt.show()
```



Разбор плотности данных (Cohesion) по рисунку для обучающих (Train) данных

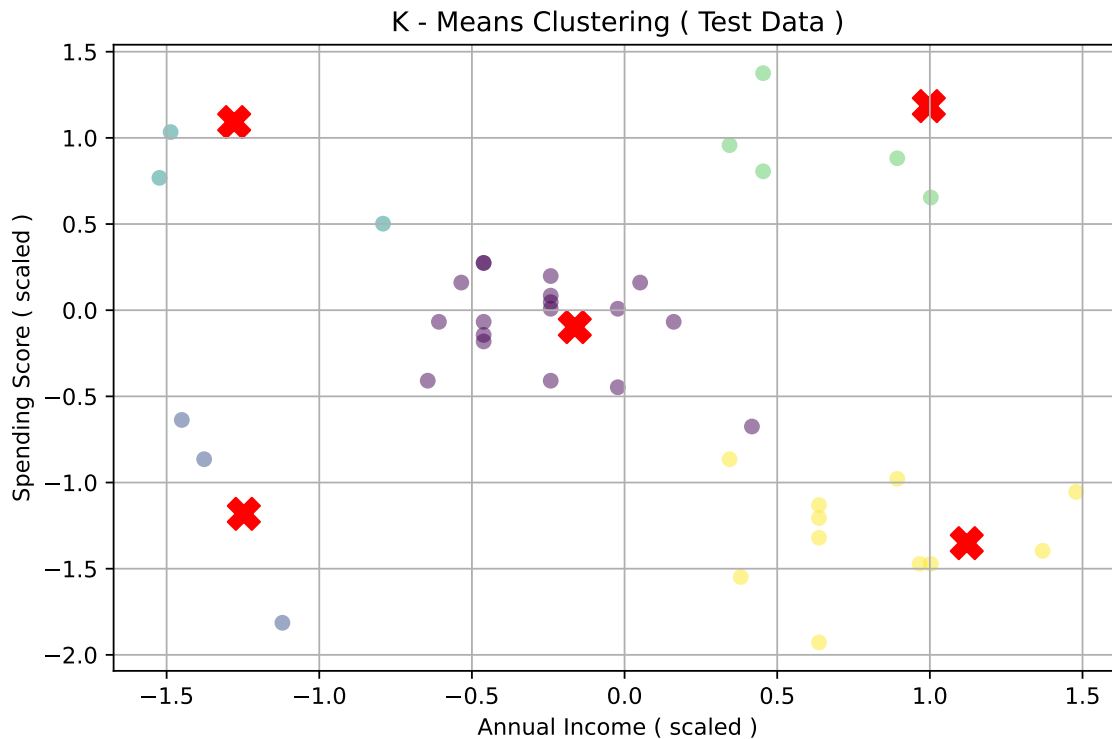
По рисунку с обучающими данными видно, что **плотность, то есть теснота внутри группы, у разных групп неодинакова**. У **средней группы, той, что около начала координат**, точки собраны вокруг центра довольно тесно, и поэтому **разброс внутри группы виден небольшой**, тогда как у групп справа сверху (высокий Annual Income и высокий Spending Score) и справа снизу (высокий Annual Income и низкий Spending Score) точки внутри одной

и той же группы разбросаны шире, и разброс внутри выходит сравнительно большим. То есть на обучающих данных с точки зрения расстояний внутри группы теснота у разных групп разная, и толковать это можно так, что средняя группа плотнее остальных.

Разбор разделения данных (**Separation**) по рисунку для обучающих (**Train**) данных

На рисунке с обучающими данными пять групп в целом занимают разные области, поэтому разделение между группами заметно. Особенно у **групп слева сверху, слева снизу, справа сверху и справа снизу границы сравнительно ясные, ведь они лежат далеко друг от друга**. Однако **средняя группа собрана около начала координат, и есть места, где расстояние до соседних групп выглядит довольно небольшим**, поэтому на отдельных границах разделение может оказаться слабее, чем у других пар групп.

```
# рисунок с результатом разбиения на проверочных данных
plt.figure(figsize=(8, 5))
# шрифт с поддержкой кириллицы, чтобы буквы в подписи не пропадали
plt.rc('font', family = "DejaVu Sans")
plt.scatter(test_scaled[:, 0], test_scaled[:, 1], c=test_labels,
            cmap="viridis", alpha=0.5)
plt.scatter(*final_kmeans.cluster_centers_.T, s=200, c="red", marker="X",
            label="Centroids (центры, полученные на обучающей части)")
# центры, найденные на обучающей части, накладываем на рисунок
# с проверочными данными, ведь по смыслу задания заново
# обучать модель на проверочной части не нужно
plt.title("K - Means Clustering ( Test Data )")
plt.xlabel("Annual Income ( scaled )")
plt.ylabel("Spending Score ( scaled )")
plt.grid(True)
plt.show()
```



Разбор плотности данных (Cohesion) по рисунку для проверочных (Test) данных

На рисунке с проверочными данными тоже видна **разница в разбросе внутри групп, а значит, и в их плотности**. У **группы, которая лежит в середине**, точки собраны в сравнительно узкой области, и **плотность выглядит высокой**, тогда как у **группы справа снизу** есть точки, разбросанные шире вдоль оси Annual Income, поэтому разброс внутри группы оказывается сравнительно большим и **плотность можно считать низкой**. Значит, и на проверочных данных теснота внутри группы у разных групп разная.

Разбор разделения данных (Separation) по рисунку для проверочных (Test) данных

Разделение данных (Separation), в отличие от плотности (Cohesion), которая смотрит, насколько тесно точки собраны внутри каждой группы, это **понятие о том, насколько ясна граница между разными группами, то есть лежат ли две группы врозь или же перекрываются и смешиваются вблизи**.

Если посмотреть на разделение по рисунку с проверочными данными точнее, жёлтая группа в целом лежит справа снизу, там, где Spending Score низкий, а лиловая группа сравнительно тесно собрана в середине, где по оси у значение Spending Score близко к нулю, поэтому **большинство точек различаются между собой вдоль оси у, то есть по Spending Score**. Если говорить точнее, многие точки жёлтой группы собраны в низкой части, где $y \approx -\$1.0$ и ниже, а лиловая группа сосредоточена примерно при $y \approx -\$0.5 \sim 0.3$, поэтому на

большей части области между двумя группами сохраняется некоторый промежуток.

Однако **есть и причина, по которой назвать разделение полностью ясным трудно.** На рисунке с проверочными данными часть жёлтых точек оказывается сравнительно близко к области лиловых точек, то есть у самой границы, и в итоге **между жёлтой и лиловой группами пустоты не остаётся, а появляется участок сближения.** Особенно бросаются в глаза жёлтые точки, у которых значение y близко к нижней границе лиловой группы (например, y около -1). То есть о нынешнем рисунке с проверочными данными точнее будет сказать так, что группы по большей части выглядят разделёнными, но часть жёлтых точек подходит к границе лиловой группы, и поэтому считать разделение полным нельзя. Итак, **на проверочных данных разделение в целом достигнуто, но есть участки, где граница размывается, ведь отдельные точки оказываются близко к области чужой группы.**

```

# смотрим вместе исходные значения проверочных данных, то есть
# до приведения к одному масштабу, и предсказанные группы
print("Показываем, как распределились группы у проверочных данных,"
      " вместе с числовыми значениями")

# берём HTML, чтобы таблица читалась удобнее
from IPython.display import HTML

# оставляем только нужные столбцы и добавляем столбец с группой
# создание новых таблиц сводим к наименьшему, то есть отбор и assign
html_table = (
    test_features[["Annual Income (k$)", "Spending Score (1-100)"]]
    .rename(columns={
        "Annual Income (k$)": "Annual Income",
        "Spending Score (1-100)": "Spending Score",
    })
    .assign(Cluster=test_labels)
    .reset_index(drop=True)
)

# единицы измерения в названиях столбцов убраны ради опрятной таблицы,
# ведь вместе с ними таблица обрезается и выглядит неаккуратно
# сами единицы мы уже узнали, когда смотрели на устройство данных

total_row_num = len(html_table)
half_row_num = (total_row_num + 1) // 2

left = html_table.iloc[:half_row_num].reset_index(drop=True)
right = html_table.iloc[half_row_num:].reset_index(drop=True)

# делить таблицу на левую и правую часть удобно, но так теряется
# наглядность, поэтому в каждой части делаем свой столбец Index
left.insert(0, "Index", range(1, len(left) + 1))
right.insert(0, "Index", range(len(left) + 1, len(left) + len(right) + 1))
# нумерация правой части продолжает нумерацию левой

# к обеим таблицам, левой и правой, применяем одно и то же
# преобразование типов
for side_table in (left, right):
    # переводим столбец с годовым доходом в числовой вид
    side_table["Annual Income"] = pd.to_numeric(side_table["Annual Income"])

    # переводим столбец с оценкой трат в числовой вид
    side_table["Spending Score"] = pd.to_numeric(side_table["Spending Score"])

    # переводим номер группы в числовой вид, а затем задаём целый

```

```
# тип Int64, который допускает пропуски
side_table["Cluster"] = pd.to_numeric(side_table["Cluster"]).astype("Int64")

# соединяем две таблицы по столбцам (axis=1) и получаем таблицу
# для сравнения
merged = pd.concat([left, right], axis=1)

# выводим таблицу в виде HTML, индекс скрываем, рамку показываем
HTML(merged.to_html(index = False, border = 1))
```

Показываем, как распределились группы у проверочных данных, вместе с числовыми значениями

Index	Annual Income	Spending Score	Cluster	Index	Annual Income	Spending Score	Cluster
1	60	52	0	21	70	29	4
2	20	79	2	22	21	35	1
3	30	4	1	23	78	22	4
4	78	1	4	24	60	40	0
5	71	11	4	25	48	59	0
6	65	50	0	26	46	56	0
7	48	47	0	27	54	57	0
8	87	13	4	28	48	48	0
9	88	13	4	29	70	77	3
10	39	65	2	30	72	34	0
11	48	50	0	31	19	72	2
12	98	15	4	32	23	29	1
13	85	75	3	33	43	41	0
14	54	52	0	34	54	54	0
15	101	24	4	35	78	17	4
16	88	69	3	36	62	56	0
17	44	50	0	37	73	88	3
18	78	20	4	38	73	73	3
19	54	41	0	39	85	26	4
20	48	59	0	40	54	53	0

Технический разбор результатов

Если учитывать смысл задания, а именно « **подготовить годовой доход (Annual Income) и оценку трат (Spending Score) покупателей торгового центра, затем способом K-Means выделить группы клиентов и численно оценить качество разбиения** », то приведённый ниже **технический разбор результатов** тоже оказывается осмысленным.

- (1) Прежде всего, при просмотре данных в первых пяти строках исправно загрузились переменные CustomerID, Gender, Age, Annual Income (в долларах США, в тысячах) и Spending Score (от 1 до 100), и для этого разбора были **выбраны только две переменные, доход и оценка трат**, по которым и шло разбиение на группы. Кроме того, K - Means работает на расстояниях, поэтому разница в масштабе двух переменных могла бы повлиять на итог, и чтобы этого избежать, перед обучением было выполнено приведение к одному масштабу через StandardScaler. Деление на обучающую и проверочную части (80 % и 20 %) даже при обучении без ответов можно толковать как способ проверить, **устойчиво ли найденные центры (centroid) ложатся на данные, которых модель не видела**. То, что на рисунках оси подписаны как (scaled), означает, что **группы сложились в пространстве приведённых к одному масштабу дохода и оценки трат**, а красные значки X на рисунке с обучающими данными показывают положение центра каждой группы.
- (2) Далее, способом локтя было прослежено, как меняется мера разброса (inertia) при $k = 1 \sim 10$, окончательное число групп было принято равным **пяти ($k = 5$)**, и на проверочных данных оценка силуэта составила **0.57**. Оценка силуэта сразу отражает и плотность внутри групп, и разделение между ними, поэтому этот итог указывает на то, что **и на проверочных данных границы между группами держатся сравнительно отчётливо**. Если посмотреть на то, как проверочные данные разошлись по группам, видна группа, собранная по признаку **высокого дохода и низких трат (например, Income от 70 до 100 , Spending от 1 до 29)**, и группа, собранная по признаку **высокого дохода и высоких трат (например, Income от 70 до 80 , Spending от 70 до 88)**, то есть видно, что **группы клиентов численно различаются по сочетанию уровня дохода и склонности к тратам**. Кроме того, на рисунке с обучающими данными каждая группа собрана вокруг своего центра, и на проверочных данных картина вышла похожей, а это подкрепляет мысль о **возможности того, что найденное строение групп клиентов одинаково приложимо и к новым наблюдениям**.

В целом этот итог можно толковать так, что порядок работы **выбор ключевых переменных (доход и оценка трат) → приведение к одному масштабу → выбор k способом локтя → обучение K-Means и оценка силуэта на проверочных данных** позволил разделить клиентов на осмысленные группы (что видно на рисунках) и **численно подтвердить качество этого разделения оценкой силуэта на проверочных данных (0.57)**. В частности, по отдельным частям видны **виды клиентов, где сочетаются высокий или низкий доход и высокие или низкие траты**, поэтому естественным продолжением будет свести воедино главные черты каждой группы (средний доход, среднюю оценку трат, размер группы и подобное) и перейти к **выбору целей продвижения или к построению стратегии работы с клиентами**.

Толкование результатов

Продолжая технический разбор с предыдущей страницы, перейдём к **толкованию результатов**, а именно « **о годовом доходе (Annual Income) и о склонности к тратам (Spending Score)** ».

Если посмотреть вместе на полученные рисунки с точками (обучающие и проверочные) и на средние значения силуэта (Train 0.55 , Test 0.57), то Annual Income (годовой доход) и Spending Score (оценка трат) связаны не единой зависимостью, при которой одно растёт вместе с другим, а вместе, соединяясь, дают то, что **клиентов удаётся довольно устойчиво разделить на пять групп разного склада**.

А именно, на рисунке с обучающими данными в **группе номер 2** отдельно стоят **те, у кого доход низкий, но оценка трат высокая**, а в **группе номер 1** **те, у кого и доход низкий, и оценка трат низкая**, и это указывает на то, что даже среди людей со сравнительно небольшим доходом склонность к тратам может отчётливо расходиться надвое. Напротив, в области высокого дохода одновременно видны **группа номер 3 с высоким доходом и высокими тратами** и **группа номер 4 с высоким доходом и низкими тратами**, и это показывает, что свести дело к «**высокий доход значит высокие траты**» нельзя, ведь и внутри людей с высоким доходом склонность к тратам расходится.

Наконец, **группа номер 0** лежит по обеим переменным в средней области, и её можно толковать как **пограничную, буферную группу, куда легко попадают точки, лежащие сравнительно близко к остальным группам**.

То, что это строение похожим образом повторяется и на рисунке с проверочными данными, а **средняя оценка силуэта держится без большой разницы, а именно Train 0.55 и Test 0.57**, до некоторой степени численно подкрепляет мысль, что описанное выше **разбиение по сочетанию дохода и склонности к тратам** не случайно совпало с одной отдельной выборкой, а **скорее всего, повторится и на новых данных**.

Этот документ подготовлен на основе формата **Quarto Markdown (.qmd)**, который позволяет объединять код **Python** и текст документа в одном файле, и выведен в формате PDF.